

By: Ray Goh

CHEATERS LEAVE FOOTPRINTS

FORENSICS OF CHEATS IN MODERN GAMES



\$ whoami



Jack Sessions

Worker at big bank corp
Youtuber/Rapper
Security Researcher
pre-seed money cs student (ie. broke)



Ray Goh

Threat Detection & IR @ Cloudflare
caffeine addict
(gacha) games
pre-seed money cs student (ie. broke)

Why is competitive cheating a forensic problem?



Competitive games are hostile multi-user environments with thousands of players and persistent sessions. This provides rich telemetry (eg. disk/network/memory), resulting in stronger forensic signals than most enterprise networks.

Cheating vs Modding: What's the difference?

Cheating

Techniques to gain an unfair advantage over other players that did not consent to playing in a modified environment

Modding

Typically modifies the game for yourself/a community without intent of gaining an unfair advantage over others (eg. new textures/maps/mechanics)

VS.



Levels of Game Cheats

Easy

01

Local File Modification

Attacker modifies files on disk that the application trusts at load time (eg. local save files/configs, inventory, map data)

02

External File Modification

Scan the game's live process memory with Cheat Engine, narrow down the game variable (eg. health/currency) , and freeze or overwrite it.

03

Botting

Automate in-game actions by simulating input (mouse/keyboard) or directly interacting with game API/protocol

04

Hooking

Inject code into the game process and intercept render or aiming function calls (eg. wallhacks)

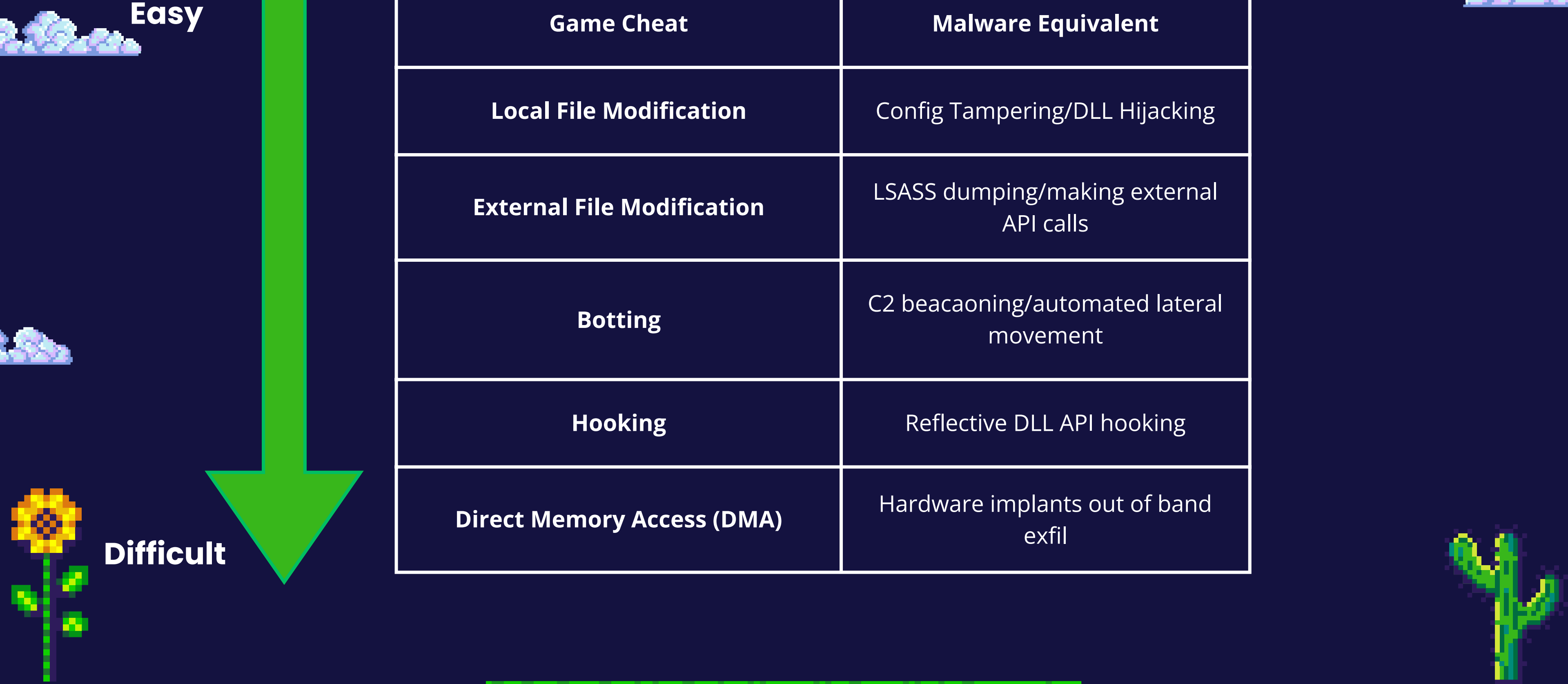
05

Direct Memory Access (DMA)

A PCIe card installed in the cheating PC lets a second machine read your game's RAM directly → no process/OS calls detectable by anti-cheats because it never touches the CPU!

Difficult

Levels of Game Cheats → Mapped to Malware

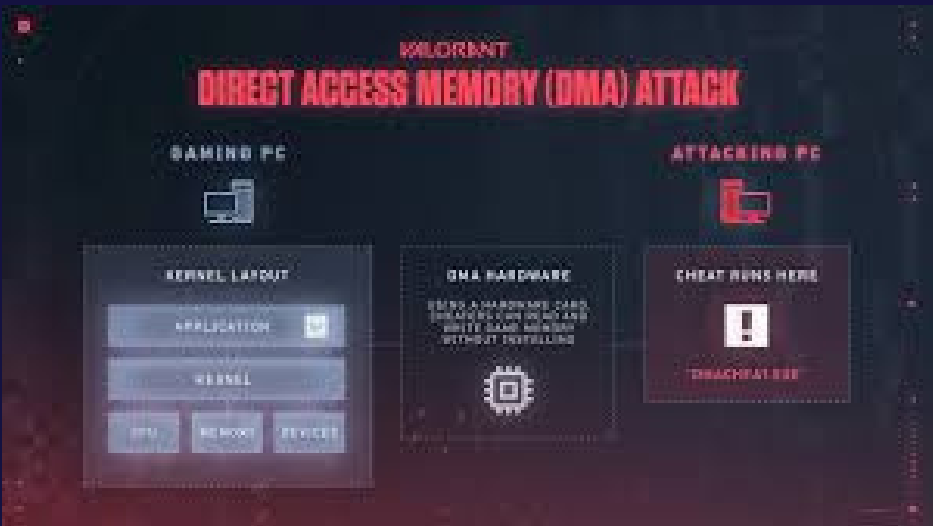


Easy

Difficult

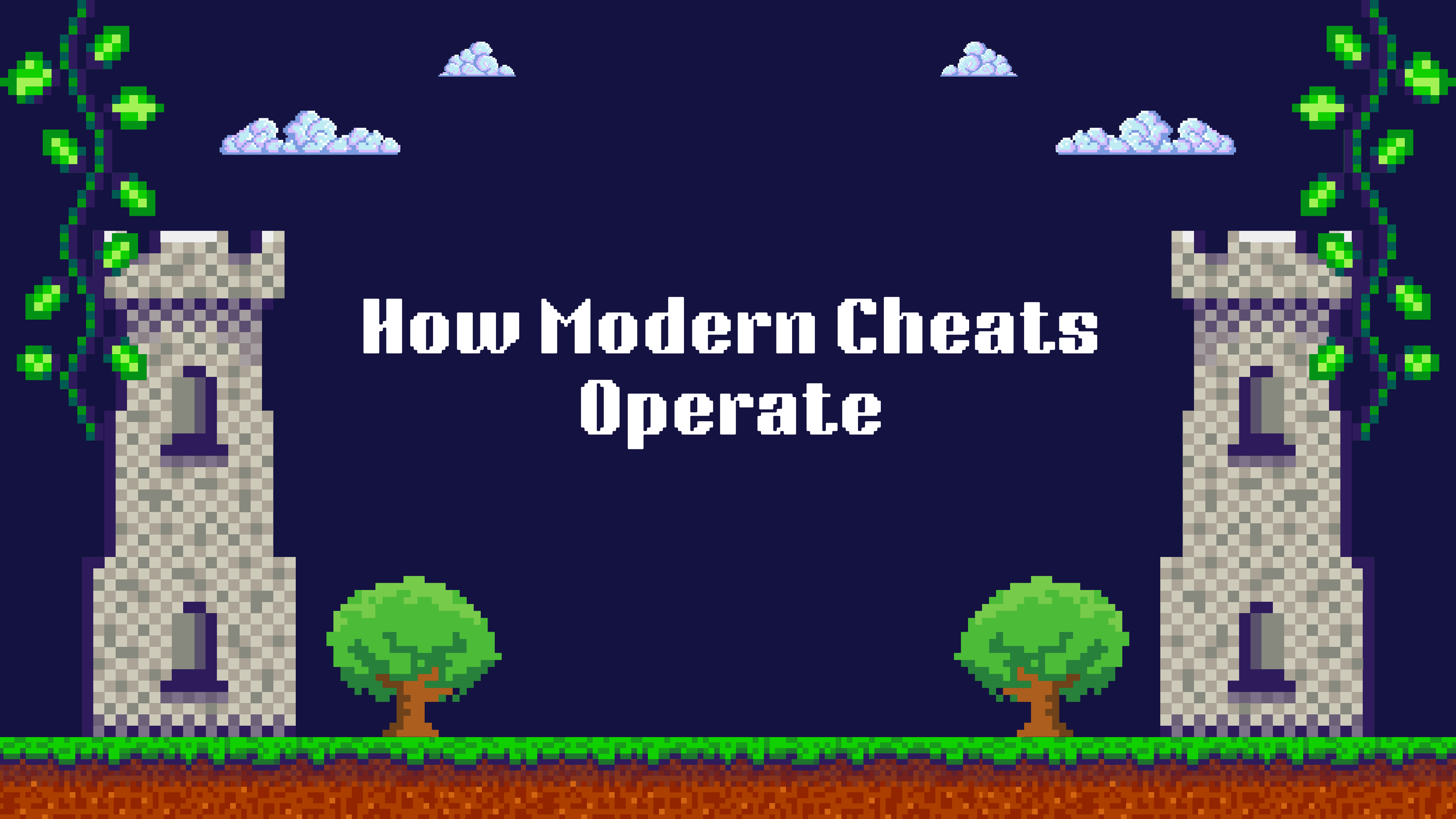
Game Cheat	Malware Equivalent
Local File Modification	Config Tampering/DLL Hijacking
External File Modification	LSASS dumping/making external API calls
Botting	C2 beacaoning/automated lateral movement
Hooking	Reflective DLL API hooking
Direct Memory Access (DMA)	Hardware implants out of band exfil

What we're looking at today



Sample	Game	Cheat Type
L2: apexdream	Apex Legends	External ESP
L4: LoL mono DLL Injection + Loader	LoL	DLL Injection ESP
L5: PCILeech DMA	Valorant	DMA

How Modern Cheats Operate



The Two-Component Model

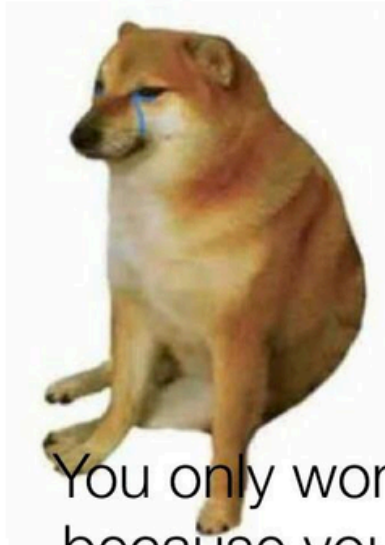
CHEAT = **BYPASS** + **PAYLOAD**

Pc gamers



We are the most accurate and best gamers and will destroy anyone who uses a controller any day of the week

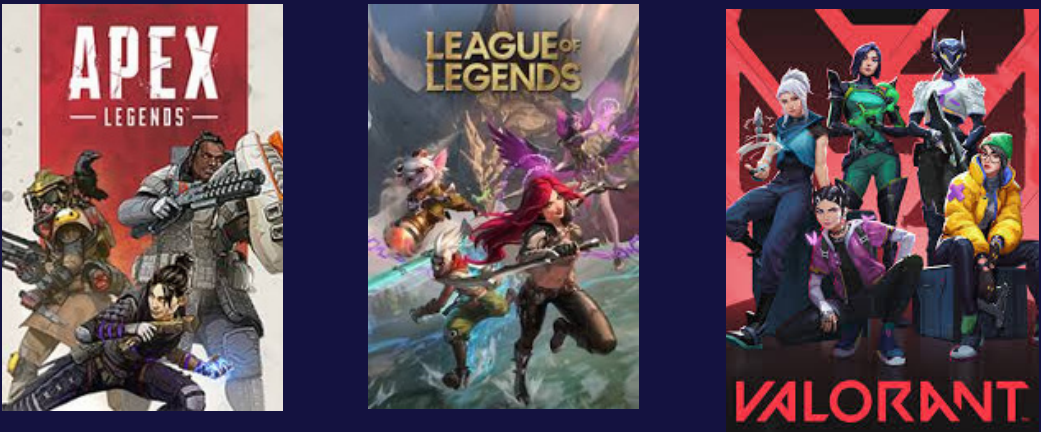
Also pc gamers



You only won because you basically have aimbot

made with mematic

Target 1



Target 2



The Two-Component Model: Sounds Familiar?

But what about malware...?

~~CHEAT~~ = **BYPASS** + **PAYLOAD**



EDR unhooking / AMSI bypass
+
C2 config / beacon
+
Implant / RAT

Cheating then

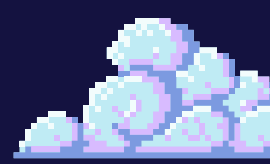
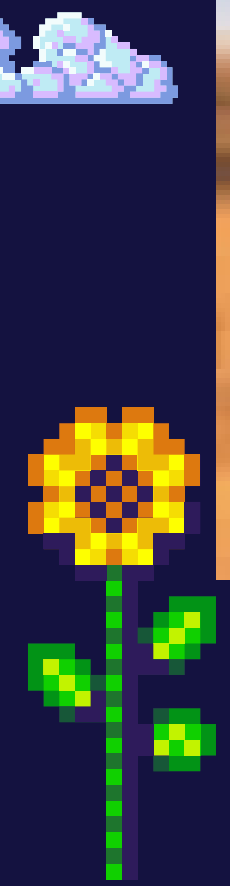


"I'm afraid you're not playing the game fairly sir. I think we should stop."

Cheating now



"Someone is slightly better than me? Wrong he's using hacks! Immediately banned!"



L2: apexdream

- **Full cheat suite:** aimbot, ESP, triggerbot, recoil control, radar through 3 Win32 API calls
 - **OpenProcess:** handle to r5apex.exe
 - **ReadProcessMemory** → entity positions, health, etc.
 - **WriteProcessMemory** → forces button states (jump/shoot/reload)

There is no bypass for EasyAntiCheat! You have to implement your own EasyAntiCheat bypass.



L4: 1v1.lol DEMON (Mono)

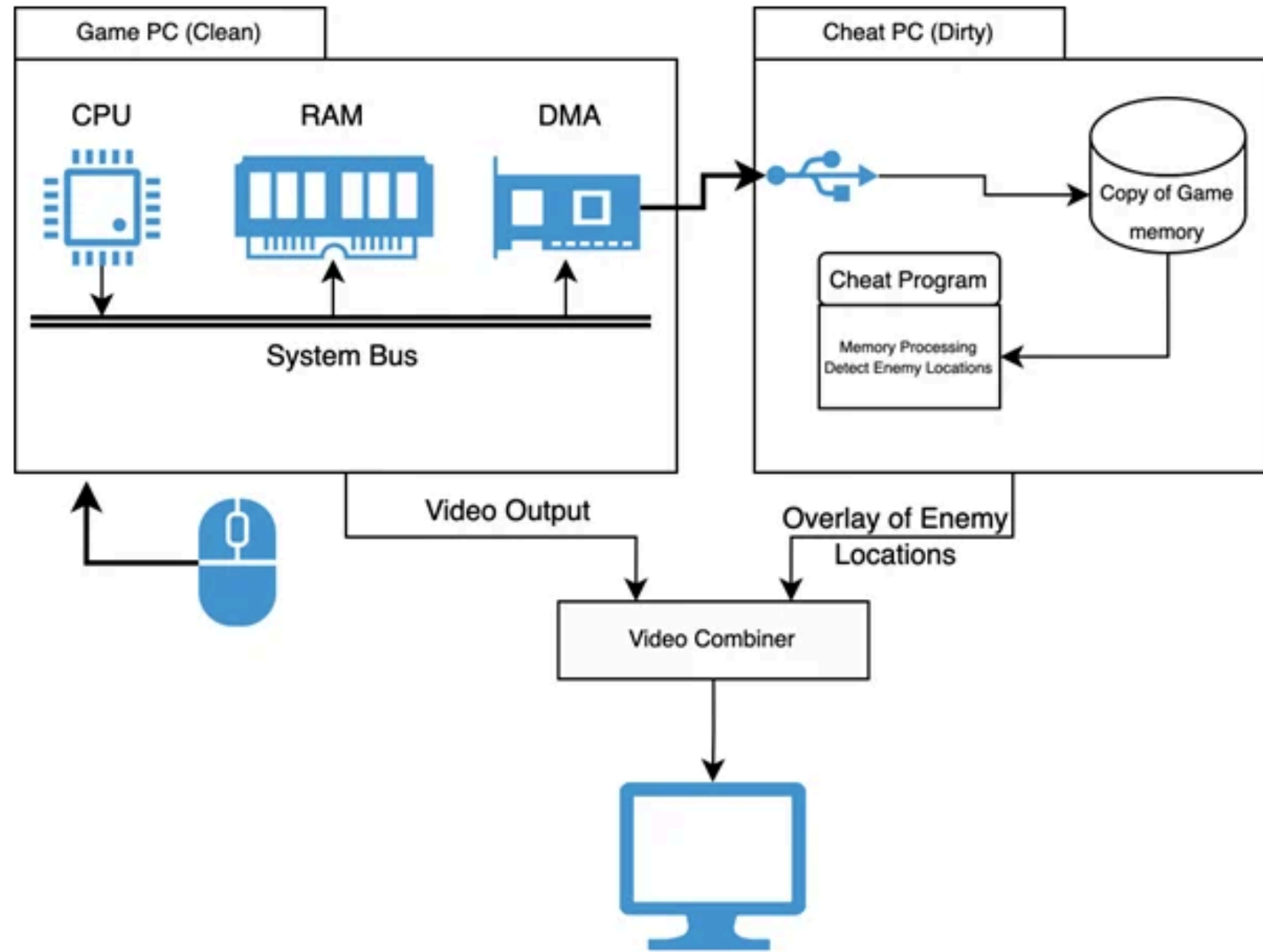
- **Full internal cheat suite for 1v1.lol:** Contains anti-cheat bypass (ported from IL2CPP → Mono) + game cheats (aimbots/ESP/Exploits/MagicBullet/Fly)
- .NET reflection to access private game fields by name, resilient to game updates unlike offset-based cheats

Two component staging

- **loader.exe:** UPX packed, patches anti cheat via dnlib.dll, injects DLL via SharpMonoInjector
- **cheat.dll:** Unobfuscated .NET, no suspicious behaviour detected
- **Injection chain:** OpenProcess → VirtualAllocEx → WriteProcessMemory → CreateRemoteThread → DEMON.Loader.Load()



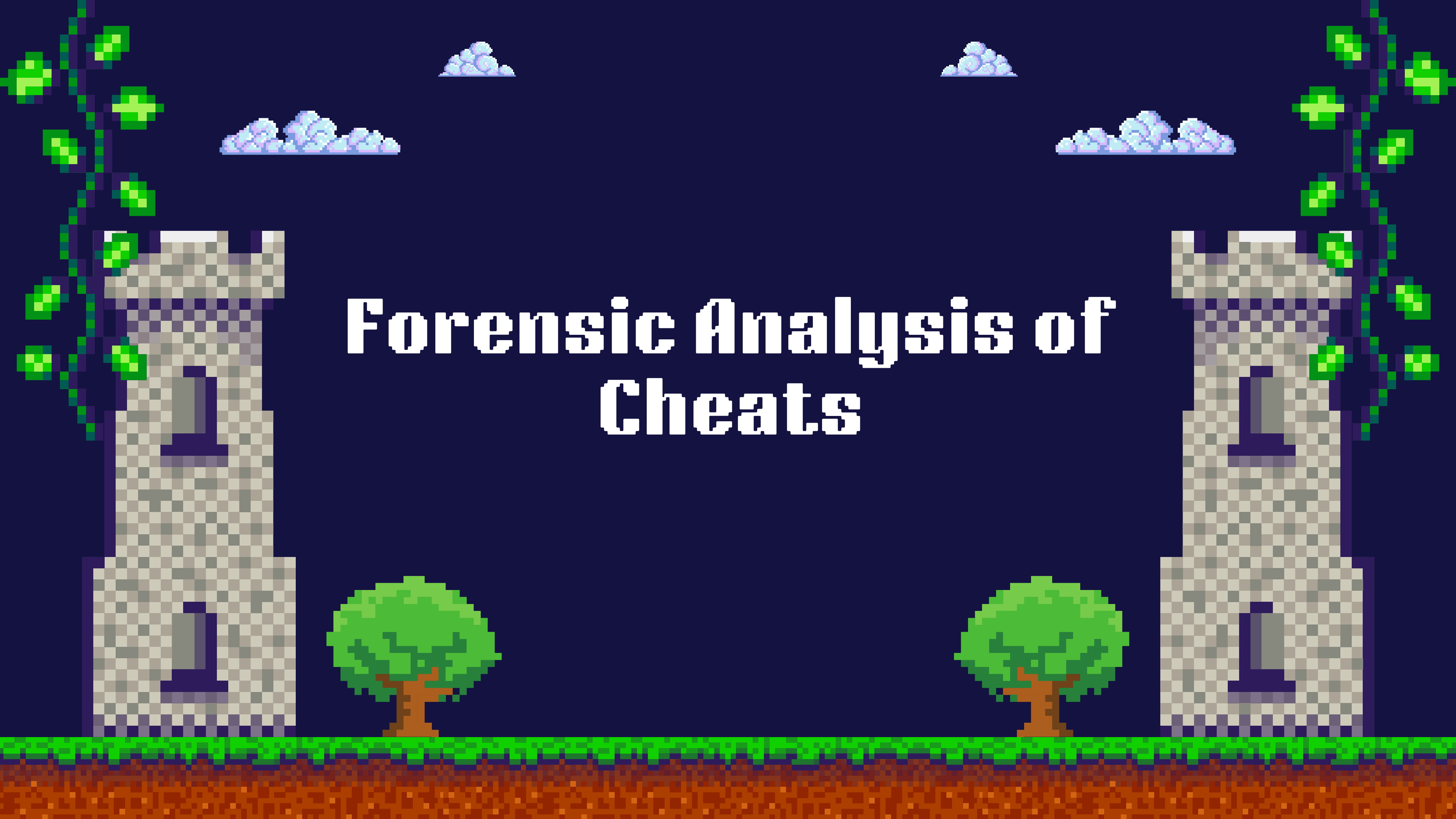
L5: Direct Memory Access (DMA) + PCIELeech



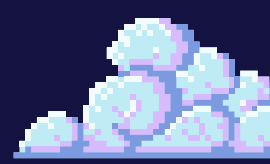
- Two physical machines, game PC has no foreign processes/injected code
- Cheat PC reads game RAM directly over PCIe hardware bus
- PCIELeech + FPGA card (~\$50) enables the read
- Overlay fed back via Video Combiner, victim screen shows enemy positions
- Zero OS-level footprint on the game machine



Forensic Analysis of Cheats

The background is a pixel art landscape. It features two grey stone towers with arched windows on either side of the title. Between the towers are two green trees with brown trunks. The sky is blue with several white, pixelated clouds. The ground is a mix of green grass and brown dirt.

Methodology: How We Analyse Each Sample



01

Static
Analysis



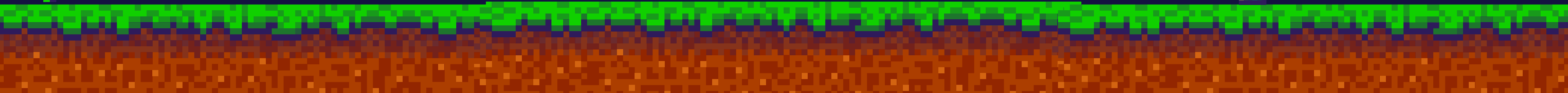
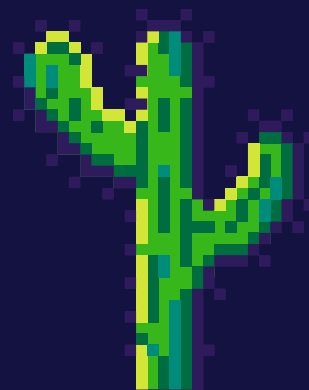
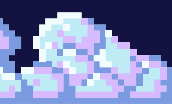
02

Dynamic
Analysis



03

Artifact
Capture



apexdream

cargo.toml

```
[dependencies]
dataview = "~1.0"
fmttools = { version = "0.1", features = ["obfstr"] }
format_xml = { version = "0.3", features = ["obfstr"] }
obfstr = "0.4"
cvar = "0.4"
intptr = { version = "0.1.6", features = ["serde", "dataview_1"] }
ini_core = "0.1.0"
bitset-core = { git = "https://github.com/CasualX/bitset-core" }
vgc = { git = "https://github.com/CasualX/vgcolors" }
named_constants = "0.2"
pelite = { version = "0.10", features = ["unstable"] }

[dev-dependencies]
external = { git = "https://github.com/CasualX/external" }
```

In-game offsets

```
[Offsets]
; v3.0.28.31
time_date_stamp=0x63fea9ae
checksum=0x22484b9
global_vars=0x1461fa0
entity_list=0x1b37b18
local_entity_handle=0x1388eb8
input=0x1ed1620
input_selected_slot=0x180
client_state=0x1462290
client_signon_state=0xAC
client_level_name=0x1C0
nst_weapon_names=0x7676c40
view_render=0x7677db8
view_matrix=0x11a210
input_system=0x14de440
input_button_state=0xB0
name_list=0xbcc7ec0
network_var_table_ptr=0xc194800
network_var_table_len=300
thirdperson_override=0x1b1c8b0
mouse_sensitivity=0x1ed55b0
fps_max=0x139a940
mp_gamemode=0x1f21c50
in_attack=0x7678368
```

config.ini

```
[ANY]
aim.trigger.enable=false
aim.trigger.radius=8.0
aim.trigger.radscale=1.0
aim.trigger.release=0.08
aim.trigger.again=0.16
```

process.rs

```
// Finally write the dump to disk
s! { let path = "/re/apex/r5apex.bin"; };
api.dump_bin(path, target);
api.log(f!("Wrote \"{path}\"!"));
```


L4: LoL DLL Injection + Loader

loader.exe: very sus

indicator (26)	detail
file > name	c:\users\hysan\downloads\loader_[unknowncheats.me]\loader.exe
file > signature	Microsoft Linker 48.0 Microsoft.NET
file > sha256	D1BD47BC8C7A262CEDAF34F6EAE05ACCFEEA18898086B2D48896FD3E...
file > info	size: 27136 bytes, entropy: 5.489
file > type	executable, 64-bit, console
virustotal > permalink	https://www.virustotal.com/gui/file/d1bd47bc8c7a262cedaf34f6eae05...
virustotal > scan-date	2025-07-29 19:51:54
virustotal > score	44/72
stamp > compiler	Sat Nov 29 05:43:07 2087
file-name > version	Loader.exe
languages > names	neutral
resources > info	count: 2, size: 1270 bytes, file-ratio: 4.68%
.NET > assembly > GUID	94499C25-9038-49CF-B82-2445EB1853E

44

/ 72

Community Score

44/72 security vendors flagged this file as malicious

d1bd47bc8c7a262cedaf34f6eae05accfee18898086b2d48896fd3ea6b7978

Loader.exe

peexe

checks-user-input

assembly

detect-debug-environment

cheats.dll: safe..?

indicator (22)	detail
file > name	c:\users\hysan\downloads\1v1 demon (mono)_[unknowncheats.me]_dll
file > signature	Microsoft Linker 48.0 Microsoft.NET
file > sha256	9B1687F0F258BD47ADA3B6D7C56E57E510B8C8C1496935CC23796B7BC4...
file > info	size: 25088 bytes, entropy: 5.590
file > type	dynamic-link-library, 64-bit, console
virustotal > permalink	https://www.virustotal.com/gui/file/9b1687f0f258bd47ada3b6d7c56e57...
virustotal > scan-date	2025-03-28 02:41:14
virustotal > score	0/73
stamp > compiler	Mon Jan 20 08:07:30 2048
file-name > version	1v1 DEMON (MONO).dll
languages > names	neutral
resources > info	count: 1, size: 860 bytes, file-ratio: 3.43%
.NET > assembly > GUID	F51290F5-9CBF-429B-9345-182249FE5BAC
file > description	1v1 DEMON (MONO)

L4: LoL DLL Injection + Loader

Notable MITRE ATT&CK Techniques observed

- T1055: Process Injection (shows up under both Privilege Escalation and Defense Evasion)
- T1497: Virtualization/Sandbox Evasion (3 techniques flagged)
- T1562:Impair Defenses

Processes Injected

 C:\Program Files\Google652_61860238\bin\updater.exe

Calls Highlighted

-  NtResumeProcess
-  NtSuspendProcess
-  GetTickCount
-  IsDebuggerPresent

Flagged imports from pestudio

imports (400)	namespace	flag (10)	type	ordinal	library
EnumProcessModules	-	x	p/Invoke	-	psapi.dll
GetModuleFileNameEx	-	x	p/Invoke	-	psapi.dll
EnumProcessModulesEx	-	x	p/Invoke	-	psapi.dll
GetModuleInformation	-	x	p/Invoke	-	psapi.dll
OpenProcess	-	x	p/Invoke	-	kernel32.dll
CloseHandle	-	-	p/Invoke	-	kernel32.dll
IsWow64Process	-	-	p/Invoke	-	kernel32.dll
WriteProcessMemory	-	x	p/Invoke	-	kernel32.dll
ReadProcessMemory	-	x	p/Invoke	-	kernel32.dll
VirtualAllocEx	-	x	p/Invoke	-	kernel32.dll
VirtualFreeEx	-	-	p/Invoke	-	kernel32.dll
CreateRemoteThread	-	x	p/Invoke	-	kernel32.dll

L4: Windows Defender Blocked Me :(

Trojan:Win32/Kepavll!rfn

Alert level: Severe

Status: Active

Date: 12/05/2026 9:52 AM

Category: Trojan

Details: This program is dangerous and executes commands from an attacker.

[Learn more](#)

Affected items:

file: C:\Users\hysan\Downloads\1v1DEMON_[unknowncheats.me]_dll

```
PS C:\WINDOWS\system32> Get-MpThreat
```

CategoryID	: 8
DidThreatExecute	: False
IsActive	: False
Resources	:
RollupStatus	: 33
SchemaVersion	: 1.0.0.0
SeverityID	: 5
ThreatID	: 2147939874
ThreatName	: Trojan:Win32/Kepavll!rfn
TypeID	: 0
PSComputerName	:

L5: The Last Artifact

404

Not Found

The resource requested could not be found on this server!

The Only Signal Is the Player

No process artifacts → detection shifts entirely to server-side behavioral telemetry

Inhuman reaction times and aim patterns are the only remaining forensic signal

This is **User and Entity Behavior Analytics (UEBA)**:
analyzing whether the entity is biologically plausible, not whether the binary is malicious



Post Incident Reconstruction

The background is a pixel art illustration. It features two tall, grey stone towers with crenellated tops and arched windows, one on the left and one on the right. Between the towers are two green, rounded trees with brown trunks. The sky is a solid blue color with several white, pixelated clouds. The ground is a mix of green grass and brown dirt.

L2: apexdream post-incident reconstruction

01

Easy anti cheat ban issued → ban timestamp as starting point

02

Prefetch: Loader binary executed before game launch, filename + load time DLLs recorded even if binary is later deleted

03

Amcache: Binary hash persists independently of the file, deletion doesn't remove it

04

During session: Sysmon EID 10, external process held open handle to r5apex.exe with PROCESS_VM_READ access logged at acquisition, not just at ban

05

On disk: config.ini + gamedata.ini present with creation timestamps correlating to session

L4: LoL DLL Post-Incident Reconstruction

01

Starting Point: Vanguard ban timestamp

02

Memory forensics: RWX region inside game process with no mapped file path, pe-sieve flags injected module even from post-execution memory dump

03

Sysmon EID 8 (CreateRemoteThread): Logged at injection time, ties loader PID to game PID

04

Disk: Loader binary in Prefetch with execution timestamp survives binary deletion

05

Amcache: DLL hash persists in AmcacheHve independently of the file being deleted

4 Different Sources:
Prefetch + Amcache +
Sysmon EIDs + RWX
memory

A pixel art landscape featuring a dark blue sky with four white, fluffy clouds. The ground is a mix of green grass and brown dirt. Two large, green, pixelated trees with brown trunks stand on either side of the center. In the center, two yellow sunflowers with green stems and leaves are positioned. The text "WHY THIS MATTERS BEYOND GAMES" is written in a bold, white, pixelated font with a pink outline, centered in the upper half of the image.

WHY THIS MATTERS BEYOND GAMES

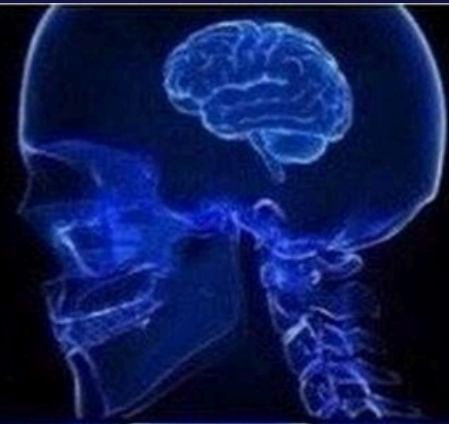
Same Playbook, Different Target

Cheat Technique	Malware/EDR Equivalent	Shared Artifact
L2: External memory read via Win32 API	LSASS credential dumping	Cross process handle in handle table
L4: DLL Injection + manual mapping	Reflective DLL injection/EDR unhooking	RWX region, VAD gap, pe-sieve patterns
L5: PCIe DMA, separate machine reads RAM	Hardware implant/out of band exfil	None on target machine, behavioural only

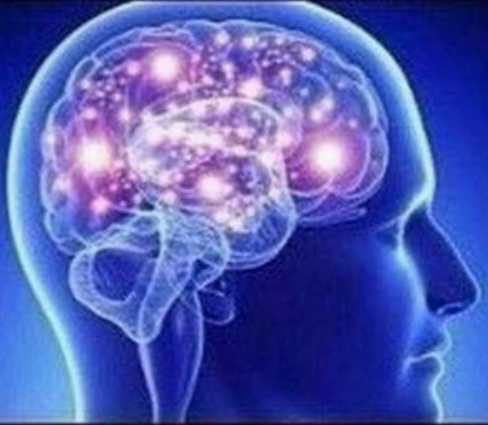


L6: CV/AI Cheats?!

INJECT
A DLL



READ MEMORY
EXTERNALLY



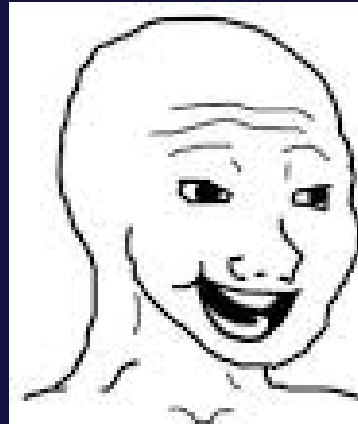
USE A
SEPARATE MACHINE



POINT A
CAMERA AT THE
SCREEN AND LET
AI FIGURE IT OUT



- Screen capture feeds a CV model which moves the mouse. No injection, no memory access, nothing for anti-cheats to find
- The process boundary that anti-cheat spent a decade defending is now completely irrelevant
- Detection has to shift from "is this binary malicious" to "is this human biologically plausible", which is just UEBA with a kill cam



Feel achieved
and supported in
developing a
game without AI
slop



Nvidia replaces it
with AI slop
anyway



THE GAME HACKER MINDSET BREAKS HERE

01

No prefetch, no amcache, no injector binary thus having to work backwards from the ban finds nothing

02

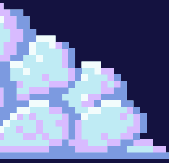
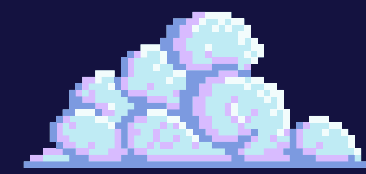
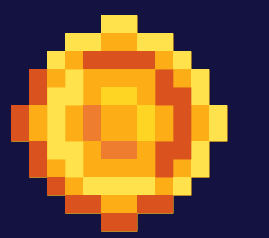
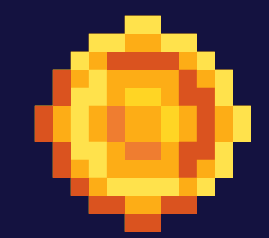
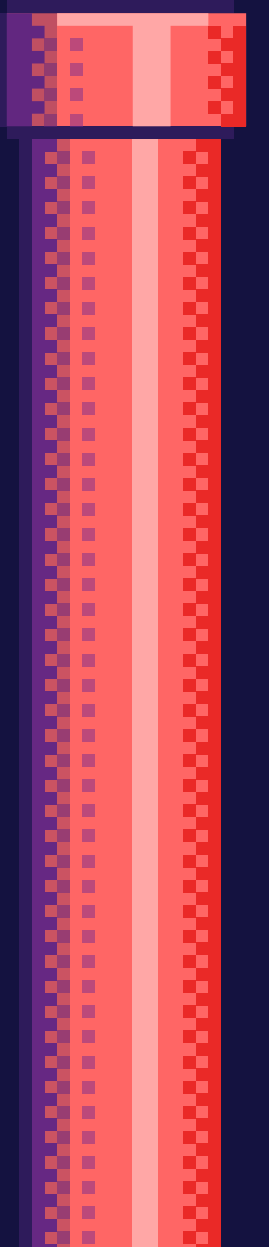
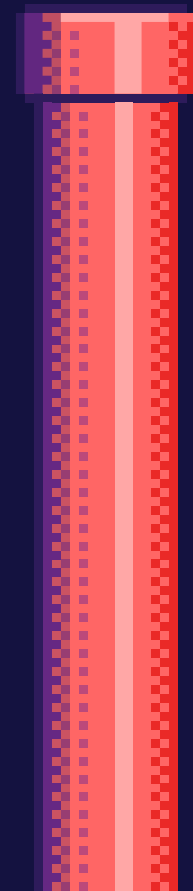
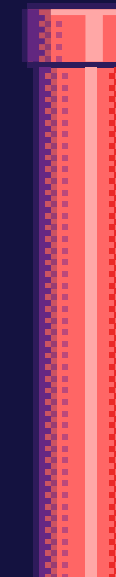
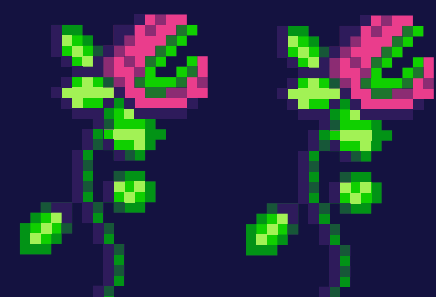
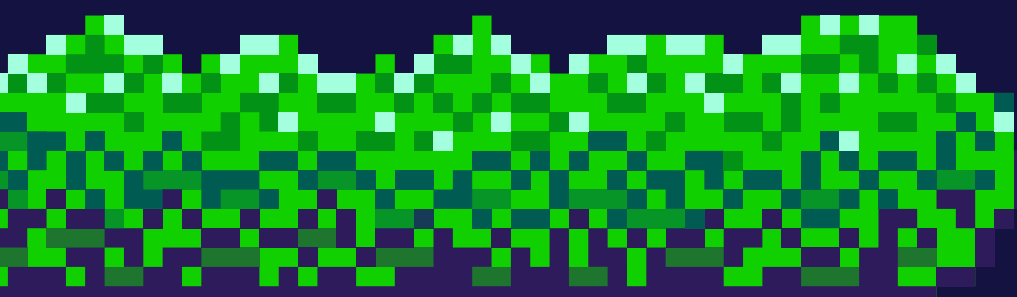
The three source correlation that worked for L2 and L4 has nothing to correlate

03

PCILeech: the cheat runs on a separate machine entirely.
ufrisk/pcileech reads physical RAM over PCIe, the game machine never sees it

04

Only evidence is server-side: aim statistics, reaction time distributions, input pattern anomalies outside human baseline



**IT WAS NEVER JUST A GAME PROBLEM
IT WAS A MINDSET ONE**



RAP

THAT'S ALL GUYS :D



Jack Sessions



Linkedin



Signal



Ray Goh



Linkedin



Signal